رواد مصر الرقمية

وزارة الاتصـــــالات
وتكنولوجيا المعلومات

Data cleaning process and organization of data with SQL

Session 3

Agenda

- Applying Basic SQL Functions for Cleaning Data
- Applying Basic SQL Queries to Organize and Filter Data

Introduction to SQL Functions for Cleaning Data

Data cleaning is a crucial step in **data analysis**, ensuring that the dataset is **accurate**, **consistent**, and **ready for further processing**. SQL (**Structured Query Language**) provides several powerful functions to help clean and standardize data efficiently.

Key SQL Functions for Data Cleaning:

TRIM, UPPER, LOWER, REPLACE, COALESCE and NULLIF
These functions are fundamental tools in ensuring that your data is **clean**, **consistent**, and ready for analysis.

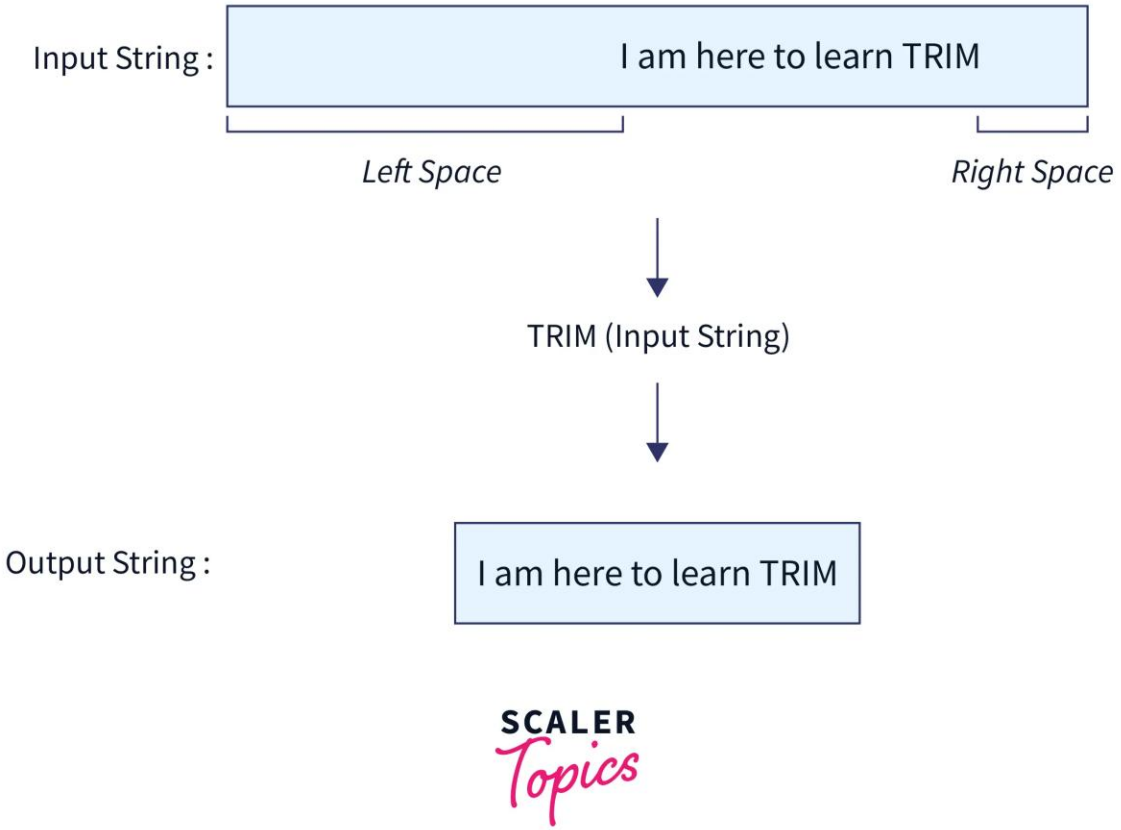
SQL Functions for Cleaning Data

TRIM: Removes **leading and trailing spaces** from a string, which is particularly useful for cleaning text fields where users may have entered data with extra spaces.

UPPER: Converts all characters in a string to **uppercase**, useful for **standardizing text data** such as ensuring all customer names are in uppercase.

LOWER: Converts all characters in a string to **lowercase**, helping to **standardize** text fields.

TRIM Function



SCALER Topics

UPPER Function

Syntax

```
UPPER(Character_Expression)
```

Example

```
Select UPPER('CONVERT This String Into upper Case')
```

Output

```
CONVERT THIS STRING INTO UPPER CASE
```

LOWER Function

Syntax

```
LOWER(Character_Expression)
```

Example

```
Select LOWER('CONVERT This String Into Lower Case')
```

Output

```
convert this string into lower case
```

SQL Functions for Cleaning Data

REPLACE: Allows you to replace occurrences of a specified substring with another substring within a string. This is commonly used for **correcting typos** or **standardizing terms** in a dataset.

COALESCE: Returns the **first non-null expression** among its arguments, allowing you to **fill in missing data** with a default value.

NULLIF: Compares two expressions and returns **NULL** if they are equal; otherwise, it returns the first expression. This is useful for **standardizing how missing** or **unknown data** is handled.

Using TRIM and UPPER for Data Cleaning

Let's walk through an example of cleaning a dataset using the TRIM and UPPER functions. Suppose you have a table employees with a column employee_name that contains names with leading and trailing spaces, and a mix of uppercase and lowercase letters.

Using TRIM and UPPER for Data Cleaning

Before Cleaning:

Retrieve the current state of the employees table:

```
SELECT * FROM employees;
```

After Cleaning Using TRIM:

Remove leading and trailing spaces from the employee_name column

```
UPDATE employees
SET employee_name = TRIM(employee_name);
```

Verifying Changes:

```
SELECT * FROM employees;
```

Using TRIM and UPPER for Data Cleaning

After Standardization Using UPPER:
Convert all names in the **employee_name** column to uppercase:

```
UPDATE employees
SET employee_name = UPPER(employee_name);
```

Verifying Changes:
Check the table to confirm that all names have been converted to uppercase:2

```
SELECT * FROM employees;
```

Using REPLACE to Standardize Data

Now, let's look at another example where we need to standardize department names in the employees table. Suppose the department names have inconsistencies, such as "HR", "Human Resources", and "Hr". We can use the REPLACE function to correct these.

Using REPLACE to Standardize Data

Before Correction:
Retrieve the current state of the employees table to view the existing data:

```
SELECT * FROM employees;
```

After Correction Using REPLACE:
Replace occurrences of 'HR' with 'Human Resources' in the department column:

```
UPDATE employees
SET department = REPLACE(department, 'HR', 'Human Resources');
```

Verifying Changes:
Check the table again to confirm that the department values have been updated:

```
SELECT * FROM employees;
```

Handling NULL Values - Using COALESCE and NULLIF

- Handling NULL values is a common task in data cleaning. Let's explore how COALESCE and NULLIF can be used to manage NULL values in your data.
- COALESCE helps fill missing values, e.g., when an employee's department is unknown, it sets the default to 'Not Assigned'.

Example 1: Using COALESCE to Handle NULL Values.

Suppose the department field in the employees' table has some NULL values that you want to replace with a default value, such as "Unknown".

Handling NULL Values - Using COALESCE and NULLIF

Before Handling NULL Values:

```
SELECT * FROM employees;
```

After Handling NULL Values Using COALESCE:

```
UPDATE employees
SET department = COALESCE(department, 'Unknown');
```

Verifying Changes:

```
SELECT * FROM employees;
```

Handling NULL Values - Using COALESCE and NULLIF

Example 2: Using NULLIF to Replace Specific Values with NULL.

Imagine that the salary field has a default value of '0' for new employees, but you want to treat '0' as NULL to indicate missing data.

These examples illustrate how COALESCE and NULLIF can be used to handle missing or default values effectively, ensuring that your data is clean and ready for analysis.

Handling NULL Values - Using COALESCE and NULLIF

Before Handling NULL Values:

```
SELECT * FROM employees;
```

After Handling NULL Values Using NULLIF:

Convert the salary field to NULL if it has a value of '0’.

```
UPDATE employees
SET salary = NULLIF(salary, 0);
```

Verifying Changes:

```
SELECT * FROM employees;
```

Advanced Data Cleaning Case Study

To consolidate your understanding, let’s work on a case study where you’ll apply multiple SQL functions to clean and standardize a complex dataset. Imagine you’re working with a customer database that contains various issues:

- Customer names with inconsistent capitalization and extra spaces.
- Department names with different abbreviations.
- Salary fields where missing data is represented inconsistently.

Advanced Data Cleaning Case Study

Step 1: Clean Customer Names:

```
UPDATE customers
SET customer_name = UPPER(TRIM(customer_name));
```

Advanced Data Cleaning Case Study

Step 2: Standardize Department Names

```
UPDATE customers
SET department = REPLACE(department, 'HR', 'Human Resources');
```

Advanced Data Cleaning Case Study

Step 3: Handle NULL and Default Values in Salary

```
UPDATE customers
SET salary = COALESCE(NULLIF(salary, '0'), 'Unknown');
```

Advanced Data Cleaning Case Study

Verifying Final Changes:

```
SELECT * FROM customers;
```

Scenario-Based Examples

Let's dive into a few more scenario-based examples to solidify your understanding of SQL functions in data cleaning. These scenarios will mimic real-world challenges that analysts face daily.

Scenario-Based Examples

Scenario 1: Cleaning Product Descriptions:

Your e-commerce database contains product descriptions that are inconsistent, with leading and trailing spaces, and inconsistent capitalization. Use the TRIM function to remove unnecessary spaces and LOWER to standardize the case of all product descriptions, ensuring uniform formatting across the database.

```
UPDATE products
SET description = LOWER(TRIM(description));
```

Scenario-Based Examples

Scenario 2: Standardizing Contact Information

In a CRM database, phone numbers are stored in various formats. You want to standardize them by removing spaces and ensuring a consistent format using REPLACE.

```
UPDATE contacts
SET phone = REPLACE(phone, ' ', '');
```

Scenario-Based Examples

Scenario 3: Handling Missing Email Addresses

Your customer database has missing email addresses. For those records, you want to set a default placeholder email using COALESCE.

```
UPDATE customers
SET email = COALESCE(email, 'noemail@example.com');
```

Introduction to Data Organization with SQL

In SQL, organizing data efficiently is critical for deriving meaningful insights from large datasets. Two powerful techniques for data organization are:

- **GROUP BY:** Allows you to group rows that have the same values in specified columns into summary rows, such as finding the sum or average of grouped data.
- **WHERE Clause with Complex Conditions:** Helps filter data based on specific criteria, which is crucial for narrowing down your data to the most relevant records.

These techniques are foundational for segmenting and analyzing data, making it easier to generate reports, insights, and actionable decisions.

Understanding the GROUP BY Clause

The GROUP BY clause is used to arrange identical data into groups. It's commonly used with aggregate functions such as COUNT, SUM, AVG, MAX, and MIN.

Example: Suppose you have a sales table that tracks sales transactions. You want to know the total sales amount for each product category.

```
SELECT category, SUM(sales_amount)
FROM sales
GROUP BY category;
```

Filtering Data with Complex WHERE Clauses

The WHERE clause is used to filter records that meet specific criteria. You can combine multiple conditions using logical operators like AND, OR, and NOT

Example: Consider a customers table where you want to filter customers who made purchases over $500 and are located in specific cities.

Steps for Using the WHERE Clause with Multiple Conditions

1- Start with Simple Filtering

Example: Filter customers with purchases over $500.

```
SELECT *
FROM customers
WHERE purchase_amount > 500;
```

Steps for Using the WHERE Clause with Multiple Conditions

2- Add a Second Condition with AND

Example: Filter customers who made purchases over $500 and are located in New York.

```
SELECT *
FROM customers
WHERE purchase_amount > 500
AND city = 'New York';
```

Steps for Using the WHERE Clause with Multiple Conditions

3-Use OR to Broaden the Filter

Example: Filter customers who made purchases over $500 or are located in either New York or Los Angeles.

```
SELECT *
FROM customers
WHERE purchase_amount > 500
OR city IN ('New York', 'Los Angeles');
```

Combining GROUP BY with HAVING for Advanced Filtering

The HAVING clause is used in combination with GROUP BY to filter groups based on aggregate values, which can't be done using the WHERE clause.

Example: You want to find product categories that have generated more than $10,000 in sales.

```
SELECT category, SUM(sales_amount)
FROM sales
GROUP BY category
HAVING SUM(sales_amount) > 10000;
```

Segmenting Data into Logical Subsets

Segmenting data involves categorizing or ranking data into meaningful subsets. SQL provides powerful tools like GROUP BY and ORDER BY to achieve this.

Example: Ranking employees based on their performance scores stored in the employees table.

```
SELECT employee_id, employee_name, performance_score
FROM employees
ORDER BY performance_score DESC;
```

Real-World Example - Customer Segmentation

Scenario: You want to segment your customers based on their total purchase amount into different categories: High-value, Medium-value, and Low-value customers.

```sql
SELECT customer_id, customer_name,
CASE
    WHEN total_purchase >= 10000 THEN 'High-value'
    WHEN total_purchase BETWEEN 5000 AND 9999 THEN 'Medium-value'
    ELSE 'Low-value'
END AS customer_segment
FROM customers;
```

Structured Exercises - Advanced Data Organization

Exercise 1: Grouping and Filtering Sales Data

Use the sales table to calculate the average sales amount for each region, but only include regions where the total sales exceed $50,000.

Solution:

```
SELECT region, AVG(sales_amount)
FROM sales
GROUP BY region
HAVING SUM(sales_amount) > 50000;
```

Structured Exercises - Advanced Data Organization

Exercise 2: Ranking Products by Popularity

Rank products based on the number of units sold, using the products table.

```sql
SELECT product_id, product_name, units_sold
FROM products
ORDER BY units_sold DESC;
```

Case Study - Organizing and Analyzing a Large Dataset

You are provided with a dataset containing sales transactions, customer information, and product details. Your tasks include:

- Grouping sales data by product category to determine total sales per category.
- Filtering the results to focus only on high-performing categories.
- Segmenting customers based on their purchase history.
- Ranking products based on sales volume.

Case Study - Organizing and Analyzing a Large Dataset

Step 1: Grouping Sales Data by Product Category

```sql
SELECT category, SUM(sales_amount) AS total_sales
FROM sales
GROUP BY category
HAVING SUM(sales_amount) > 10000;
```

Case Study - Organizing and Analyzing a Large Dataset

Step 3: Segmenting Customers Based on Purchase History

```
SELECT customer_id, customer_name,
CASE
    WHEN total_purchase >= 10000 THEN 'High-value'
    WHEN total_purchase BETWEEN 5000 AND 9999 THEN 'Medium-value'
    ELSE 'Low-value'
END AS customer_segment
FROM customers;
```

Case Study - Organizing and Analyzing a Large Dataset

Step 4: Ranking Products Based on Sales Volume

```sql
SELECT Products.ProductName, SUM(OrderDetails.Quantity) AS units_sold
FROM Products
INNER JOIN OrderDetails ON Products.ProductID = OrderDetails.ProductID
GROUP BY Products.ProductName
ORDER BY units_sold DESC;
```

Let’s Practice

Exercise #1

Write a solution to find the average selling price for each product. average_price should be rounded to 2 decimal places.

Return the result table in any order.

Table: `Prices`

Column Name	Type
product_id	int
start_date	date
end_date	date
price	int

(product_id, start_date, end_date) is the primary key (combination of columns with unique values) for this table.
Each row of this table indicates the price of the product_id in the period from start_date to end_date.
For each product_id there will be no two overlapping periods. That means there will be no two intersecting periods for the same product_id.

Table: `UnitsSold`

Column Name	Type
product_id	int
purchase_date	date
units	int

This table may contain duplicate rows.
Each row of this table indicates the date, units, and product_id of each product sold.

Example

Input:

Prices table:

product_id	start_date	end_date	price
1	2019-02-17	2019-02-28	5
1	2019-03-01	2019-03-22	20
2	2019-02-01	2019-02-20	15
2	2019-02-21	2019-03-31	30

UnitsSold table:

product_id	purchase_date	units
1	2019-02-25	100
1	2019-03-01	15
2	2019-02-10	200
2	2019-03-22	30

Output:

product_id	average_price
1	6.96
2	16.96

Explanation:

Average selling price = Total Price of Product / Number of products sold.
Average selling price for product 1 = ((100 * 5) + (15 * 20)) / 115 = 6.96
Average selling price for product 2 = ((200 * 15) + (30 * 30)) / 230 = 16.96

Exercise #2

Column Name	Type
article_id	int
author_id	int
viewer_id	int
view_date	date

There is no primary key (column with unique values) for this table, the table may have duplicate rows.
Each row of this table indicates that some viewer viewed an article (written by some author) on some date.
Note that equal author_id and viewer_id indicate the same person.

Write a solution to find all the authors that viewed at least one of their own articles.

Return the result table sorted by `id` in ascending order.

The result format is in the following example.

Example

Input:

Views table:

article_id	author_id	viewer_id	view_date
1	3	5	2019-08-01
1	3	6	2019-08-02
2	7	7	2019-08-01
2	7	6	2019-08-02
4	7	1	2019-07-22
3	4	4	2019-07-21
3	4	4	2019-07-21

Output:

id
4
7

Any Questions ?

Thank You